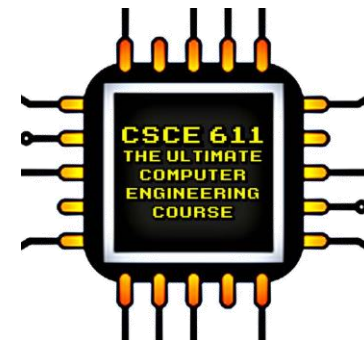


CSCE 611

Advanced Digital Design

Introduction and RISC-V ISA



South Carolina

CSCE 611 Objectives

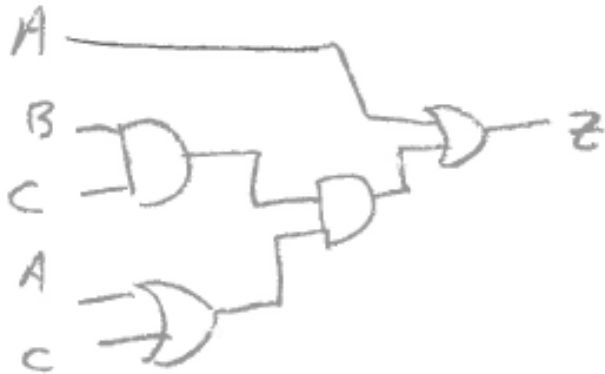
1. Apply CSCE 211 concepts at a larger scale
 - Designs consisting of 1000s of logic gates
2. Design and test a working CPU from scratch

"CSCE 211 Design Flow"

Step 1: Define behavior
(write Boolean equation)

$$Z = A + (B \cdot c) \cdot (A + c)$$

Step 2: "Synthesize" equation into a
gate-level schematic (structure)

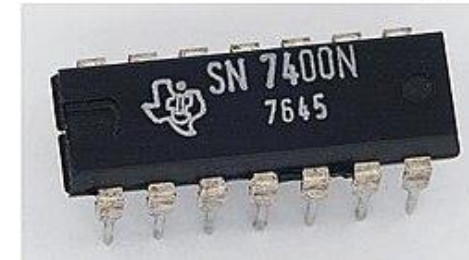
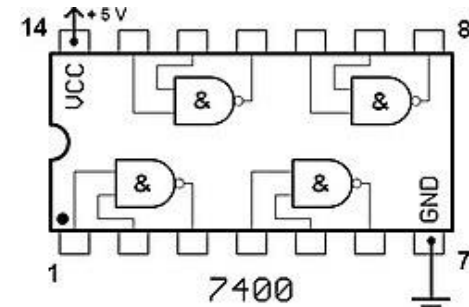


Step 3: "Map" the structure
into fewer, smaller gates

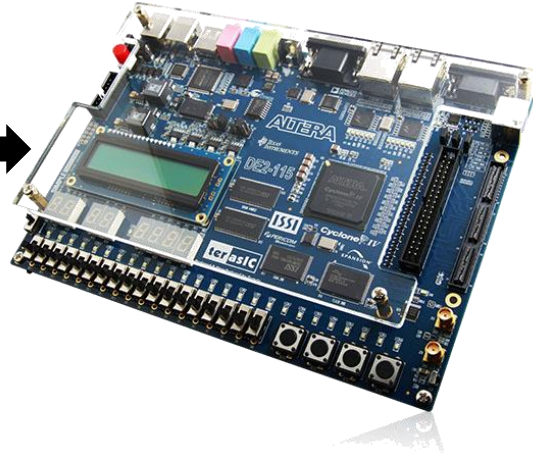
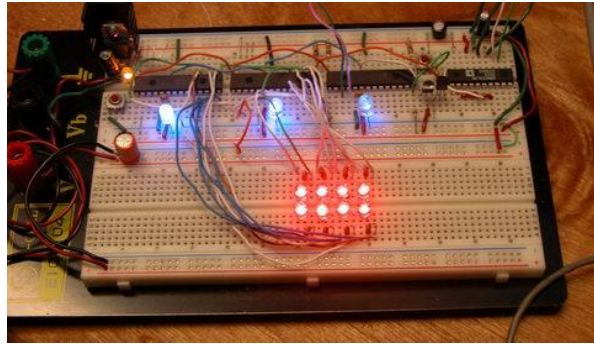
$$\begin{aligned} Z &= A + (ABC + cBc) \\ &= A + (ABC + Bc) \\ &= A + Bc(A + 1) \\ &= A + Bc \end{aligned}$$



Step 4: Place each gate onto
a "slot" on a 7400-series chip
(place-and-route)



CSCE 611 Design Flow



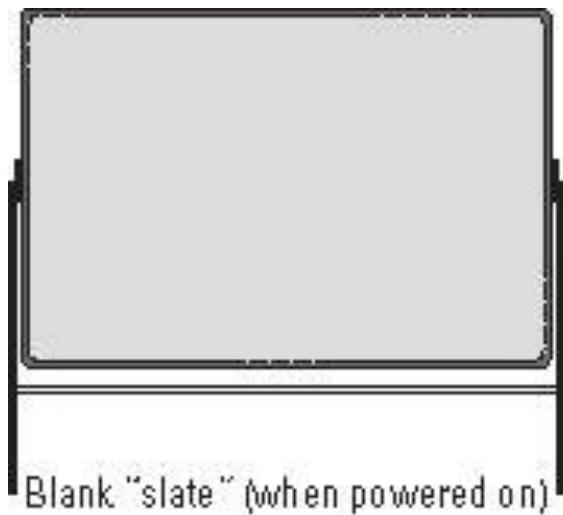
Field Programmable Gate Array (FPGA)

Hardware Description Language (HDL)


$$\begin{aligned} Z &= A + (ABC + CBc) \\ &= A + (ABC + Bc) \\ &= A + Bc(A + 1) \\ &= A + Bc \end{aligned}$$


```
module top(input logic clk,a,b,c
           output logic z);
    logic ns,cs;
    assign ns = (cs&a)~&b;
    assign z = b~&c;
    always_ff @(posedge clk)
        cs <= ns;
endmodule
```

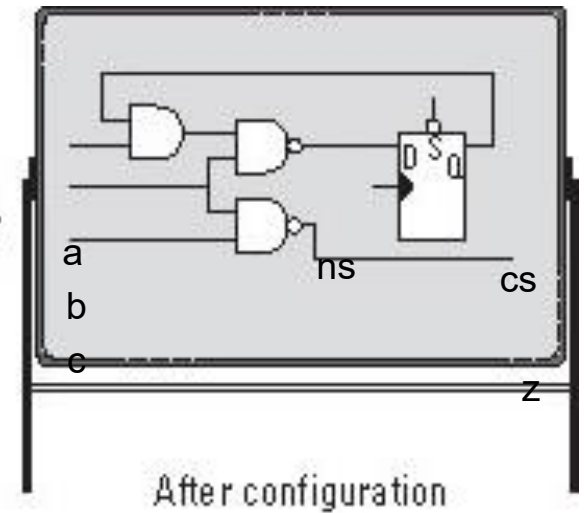
FPGA



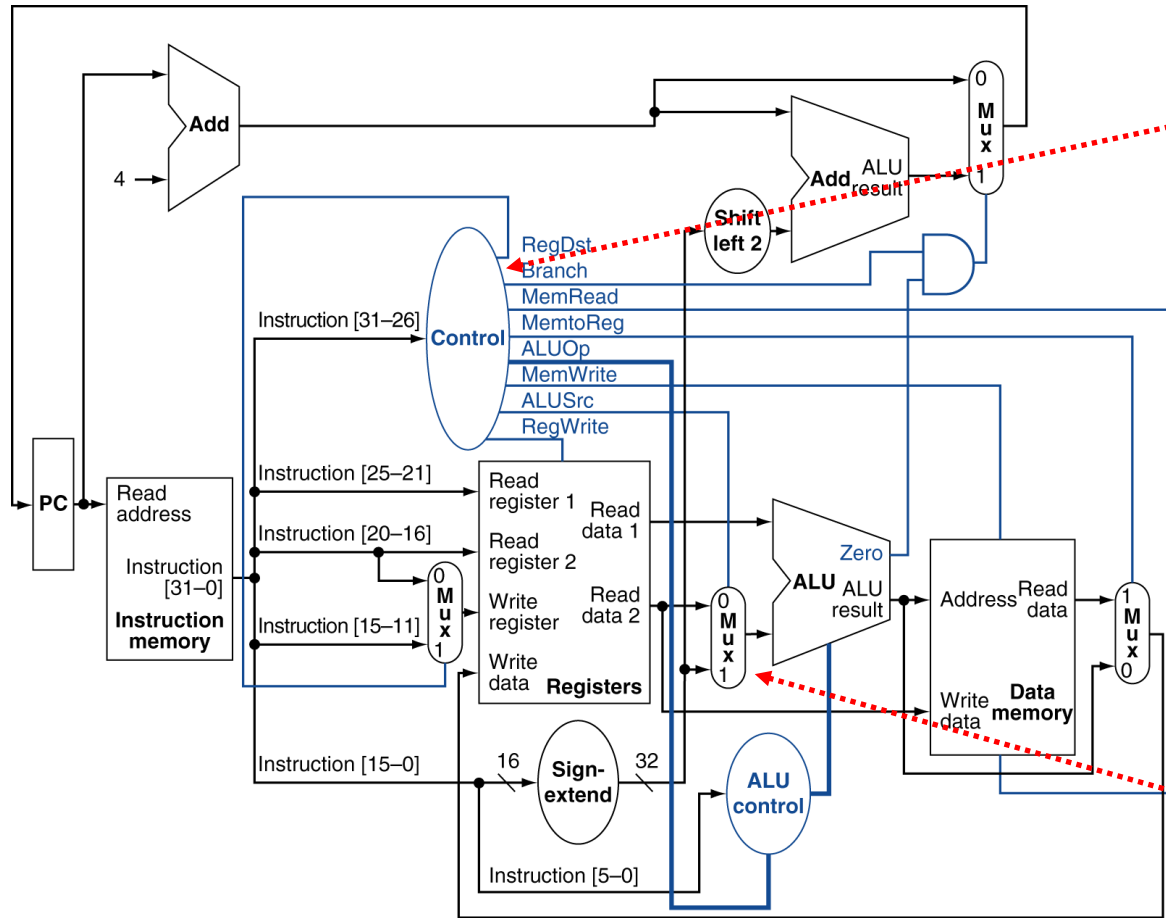
```
module top(input logic clk,a,b,c
           output logic z);
  logic ns,cs;
  assign ns = (cs&a)~&b;
  assign z = b~&c;
  always_ff @(posedge clk)
    cs <= ns;
endmodule
```



Synthesize,
map,
place-and-route



Build a CPU



structural HDL (top-down)

```
// Instantiate our control unit
control_unit my_control_unit(.opcode_i(opcode),
    .funct3_i(funct3),
    .funct7_i(funct7),
    .cssrw_i(cssrw),
    .alusrc_o_EX(alusrc_EX),
    .gpio_we_o(gpio_we),
    .regwrite_o_EX(regwrite_EX),
    .regsel_o_EX(regsel_EX),
    .aluop_o_EX(aluop_EX),
    .stall_o(stall_FCH),
    .stall_i(stall_EX),
    .pcsrc_o(pcsrc_EX),
    .aluoutput_r_i(aluoutput_r_EX));
```

behavioral HDL (bottom-up)

```
B_mux
assign B_EX = alusrc_EX == 1'b0 ?
    readdata2_EX :
    immediate_EX;
```

Performance (vs. VAX-11/780)

Year	Processor / Configuration	Performance (vs. VAX-11/780)
1978	AX-11/780, 5 MHz	1
1986	VAX 8700, 22 MHz	5
1987	Sun-4/260, 16.7 MHz	9
1988	MIPS M/120, 16.7 MHz	13
1989	MIPS M2000, 25 MHz	18
1990	IBM RS6000/540, 30 MHz	24
1991	HP 9000/750, 66 MHz	51
1992	Digital 3000 AXP/500, 150 MHz	80
1993	IBM POWERstation 100, 150 MHz	117
1994	Digital Alphastation 4/266, 266 MHz	183
1995	Digital Alphastation 5/300, 300 MHz	280
1996	Digital AlphaStation 5/500, 500 MHz	481
1997	AlphaServer 4000 5/600, 600 MHz	649
1998	Digital AlphaServer 8400 6/575, 575 MHz	993
1999	Professional Workstation XP1000, 667 MHz	1,267
2000	Intel VC820 motherboard, 1.0 GHz Pentium III processor	1,779
2001	IBM Power4, 1.3 GHz	3,016
2002	Intel D850EMVR motherboard (3.06 GHz, Pentium 4 processor with Hyper-Threading Technology)	4,195
2003	Intel Xeon EE 3.2 GHz	6,043
2004	AMD Athlon, 2.6 GHz	6,681
2005	AMD Athlon 64, 2.8 GHz	7,108
2006	Intel Core Duo Extreme 2 cores, 3.0 GHz	11,865
2007	Intel Core 2 Extreme 2 cores, 2.9 GHz	14,387
2008	Intel Xeon 4 cores 3.6 GHz (Boost to 4.0 GHz)	19,484
2009	Intel Xeon 6 cores, 3.3 GHz (boost to 3.6 GHz)	24,129
2010	Intel Core i7 4 cores 3.4 GHz (boost to 3.8 GHz)	31,999
2011	Intel Xeon 4 cores 3.6 GHz (Boost to 4.0 GHz)	34,967
2012	Intel Core i7 4 cores 4.0 GHz (Boost to 4.2 GHz)	39,419
2013	Intel Core i7 4 cores 4.2 GHz (Boost to 4.5 GHz)	40,967
2014	Intel Xeon 4 cores 3.7 GHz (Boost to 4.1 GHz)	49,870
2015	Intel Xeon 4 cores 3.6 GHz (Boost to 4.0 GHz)	49,935
2016	Intel Xeon 4 cores 3.6 GHz (Boost to 4.0 GHz)	49,935
2017	Intel Xeon 4 cores 3.6 GHz (Boost to 4.0 GHz)	49,935
2018	Intel Xeon 4 cores 3.6 GHz (Boost to 4.0 GHz)	49,935

Growth Rates and Doubling Times:

- 1978 - 1986: 25%/year (doubles every 3.1 years)
- 1986 - 2003: 52%/year (doubles every 1.7 years)
- 2003 - 2011: 23%/year (doubles every 3.3 years)
- 2011 - 2016: 12%/year (doubles every 6.1 years)
- 2016 - 2018: 3.5%/year (doubles every 20 years)

New Capabilities Enabled by Specialized Hardware

- Tesla Autopilot (2014)
Mobileye EyeQ3



- 4K video on a phone (2015)
Media Engine



iPhone 6 S
The only thing that's changed is everything.
4K
videos

- FaceID (2017)
Secure Enclave



- Animojis (2017)
Apple GPU



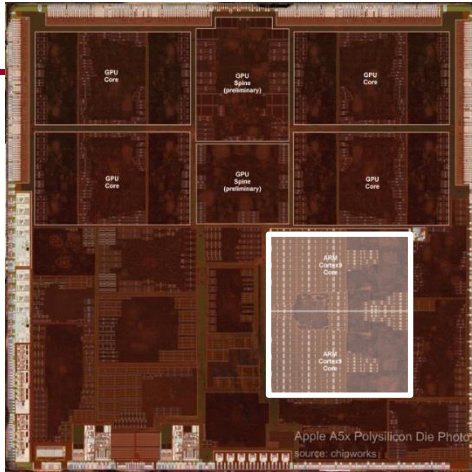
- Computational photography (2018)
Image Signal Processor



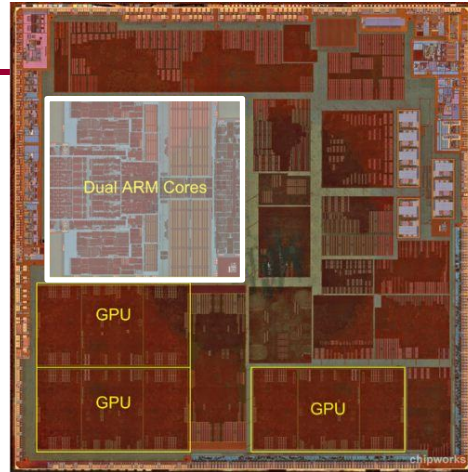
- Live Translation (2025)
Neural Engine



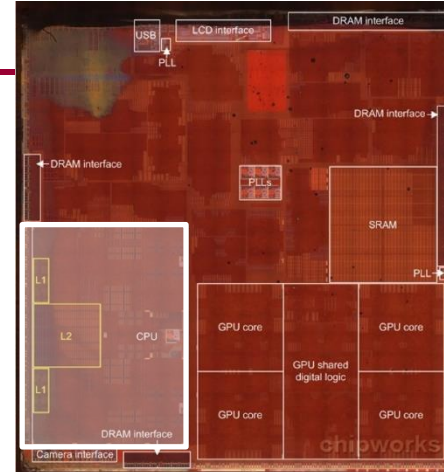
Specialization



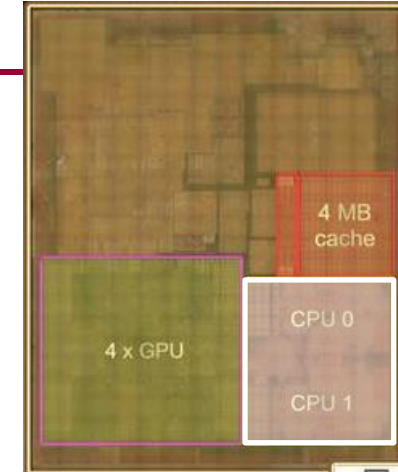
Apple A5x
(2012)



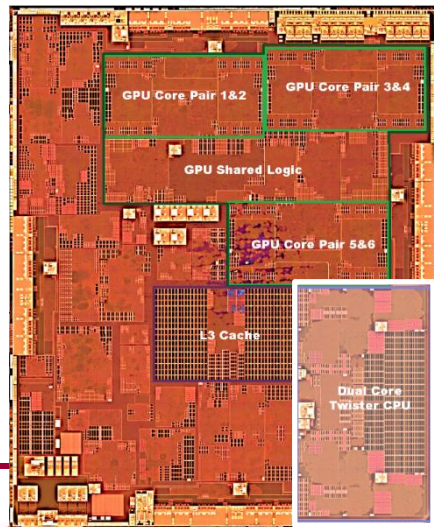
Apple A6
(2012)



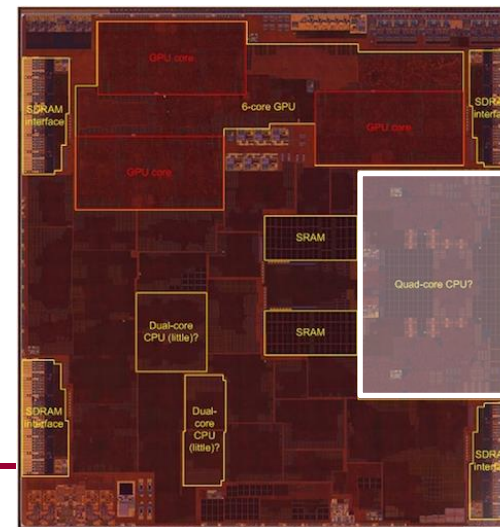
Apple A7
(2013)



Apple
A8
(2014)



Apple A9
(2015)

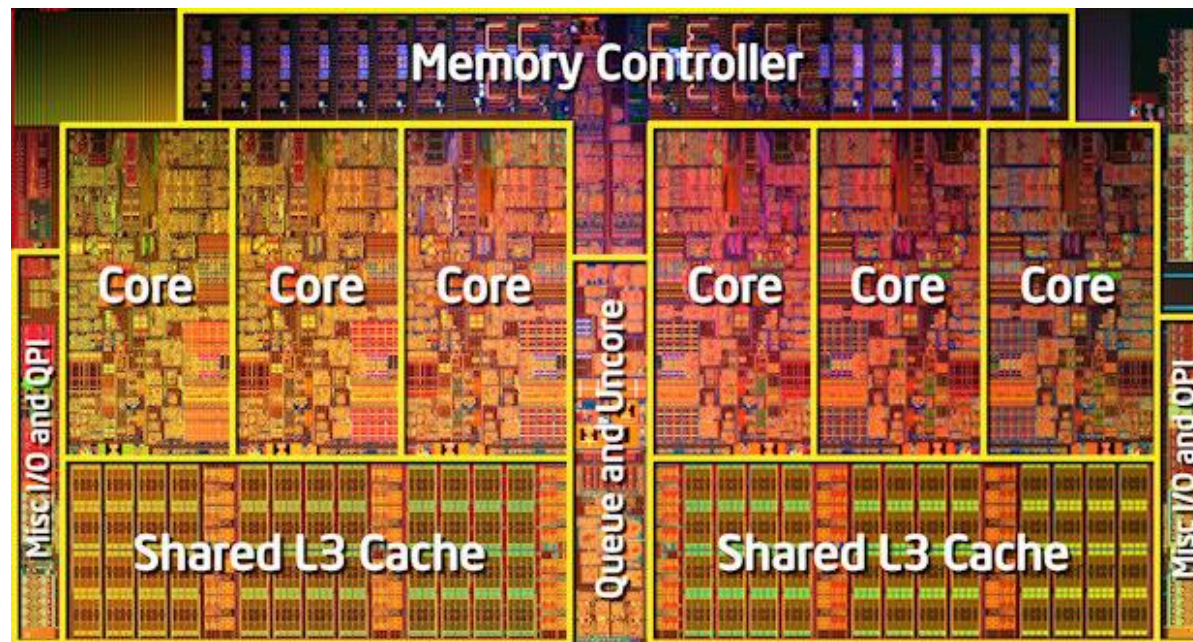


Apple A10
(2016)

CPU: ~12%

Performance: Intel Desktop Peak Performance

Desktop Processor	Tech	Year	Max. Clock Speed (GHz)	Integer IPC/core	Cores	Max. DRAM Bandwidth (GB/s)	SP Floating Point (Gflop/s)	L3 cache (MiB)
Westmere i7-620	32	2010	3.33	4	2	17.1	107	4

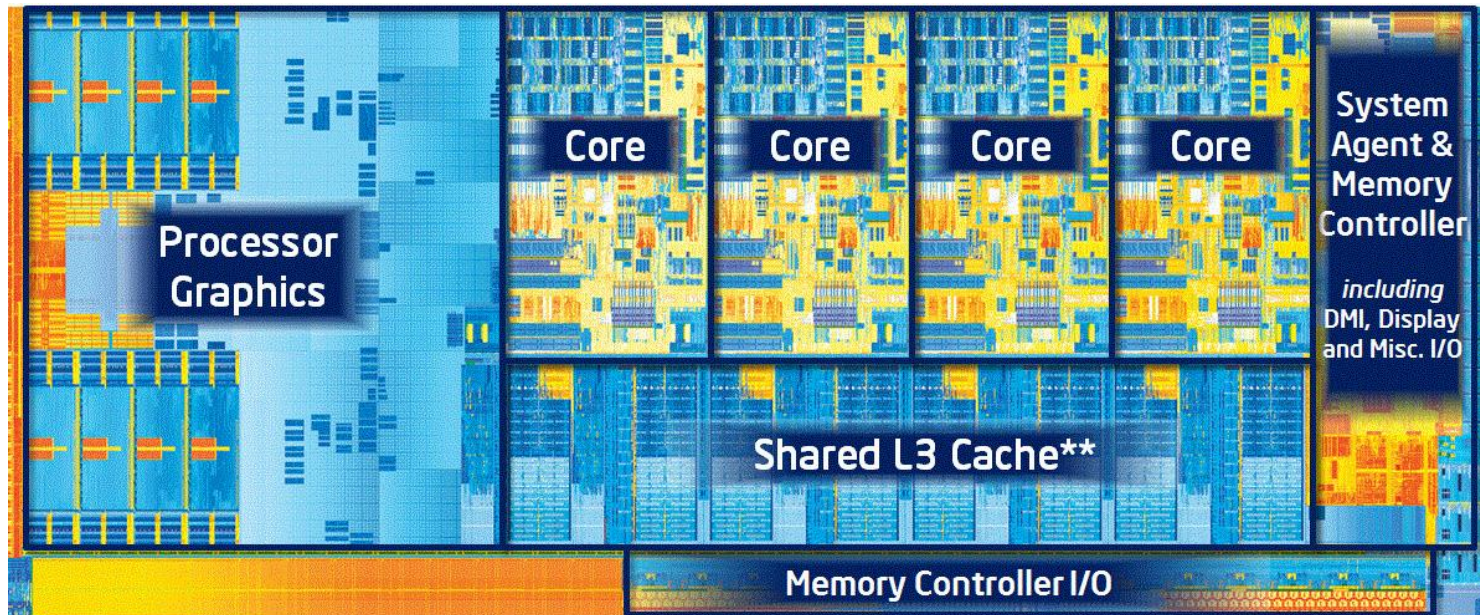


114 mm² die

CPU&cache
= 72%
(82 mm²)

Performance: Intel Desktop Peak Performance

Desktop Processor	Tech	Year	Max. Clock Speed (GHz)	Integer IPC/core	Cores	Max. DRAM Bandwidth (GB/s)	SP Floating Point (Gflop/s)	L3 cache (MiB)
Ivy Bridge i7-3770K	22	2013	3.90	6	4	25.6	250	8

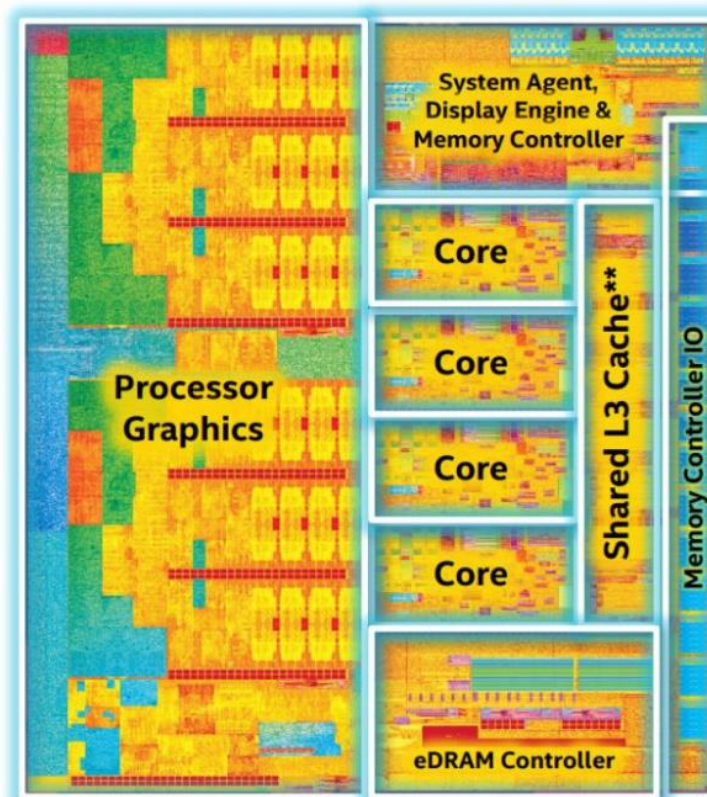


160 mm²
die

CPU &
cache =
47%
(75 mm²)

Performance: Intel Desktop Peak Performance

Desktop Processor	Tech	Year	Max. Clock Speed (GHz)	Integer IPC/core	Cores	Max. DRAM Bandwidth (GB/s)	SP Floating Point (Gflop/s)	L3 cache (MiB)
Broadwell i7-6700K	14	2015	4.20	8	4	34.1	269	8

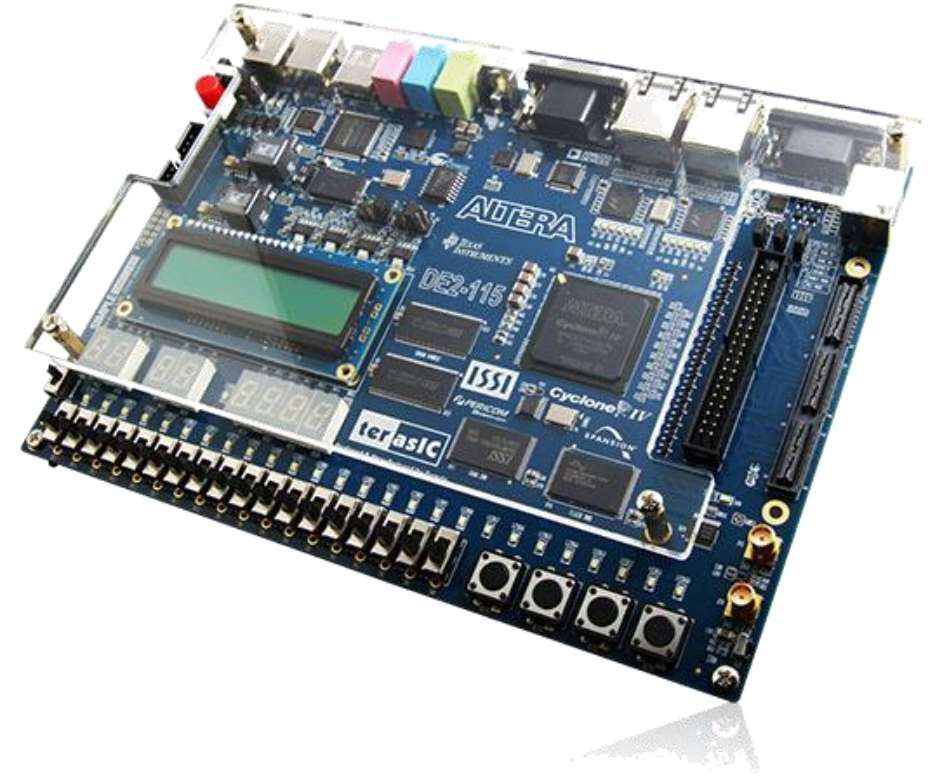


133 mm² die

CPU&cache = 20%
(27 mm²)

This Course

- In lab: design a RISC-V CPU from scratch
- Deploy on FPGA
- Learn for Lab (weeks 1 to 10):
 1. Architecture
 2. Microarchitecture
 3. Large-scale digital design and test
 4. SystemVerilog HDL
- Learn concepts (weeks 11 to 15):
 1. Sequential logic
 2. Digital building blocks
 3. Timing analysis
 4. CMOS logic



DE2 board: Intel Cyclone 4
FPGA with 115K gates

Overview

- In this lecture:
 1. Review basic architecture concepts
 2. Review assembly language programming
 3. RISC-V instruction set
 4. Fixed-point arithmetic
 5. Lab 1

Architecture vs Microarchitecture (CSCE 212)

The "native programming language" of the CPU hardware

Architecture

Assembly language program (for MIPS)

```
swap:
    muli $2, $5, 4
    add $2, $4, $2
    lw $15, 0($2)
    lw $16, 4($2)
    sw $16, 0($2)
    sw $15, 4($2)
    jr $31
```

← without knowing this

Assembler

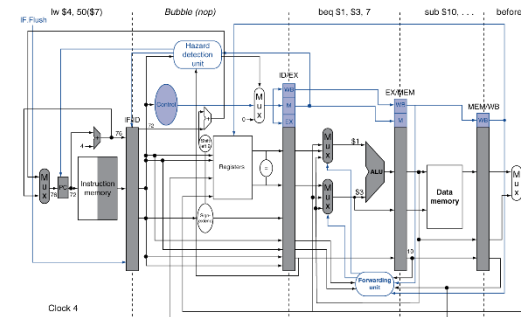
Binary machine language program (for MIPS)

```
000000001010000100000000000011000
000000000000110000001100000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
0000001111100000000000000001000
```

Software/Hardware Interface

How the ISA is implemented "under the hood"

Microarchitecture



← you can't build this...

Instructions

- From the software prospective, smallest unit of workload for processor
- High level languages need to be translated into instructions:
 - Line of C/C++ code => ~1 to 10 instructions
 - Line of Java code => ~10 to 100 instructions
 - Line of Python code => ~100 to 1000 instructions
- Instructions are:
 1. Identified by their mnemonic
 - Examples: "add", "sub", "jump"
 2. Generally limited to two inputs and one output
 3. Can only do one of three things:
 - 2-input arithmetic or logical operation
 - Bring in one word from memory and save one word to memory
 - Make a decision with regard to where to find the next instruction

RISC-V

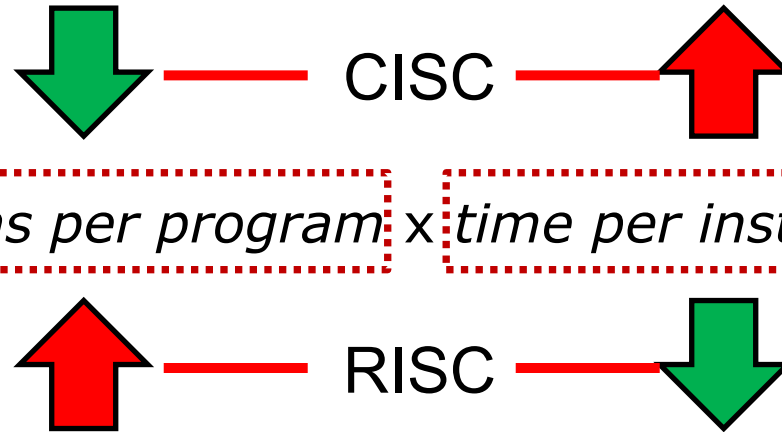
- Similar to MIPS, which you learned in CSCE 212
 - RISC-V fixes MIPS annoyances, e.g.:
 - The destination register address is always in the same spot in the instruction
 - No "shamt" field
 - No zero extension of immediates
 - All jumps generate a return address
 - RISC-V retains some MIPS annoyances, e.g.
 - No index registers
 - No autoincrement base registers
 - Store instructions place destination on right, while all other instructions on left
 - RISC-V adds some complexity
 - Increased number of encoding formats

What is RISC?

- Design “philosophies” for ISAs: RISC vs. CISC
 - CISC = Complex Instruction Set Computer
 - RISC = Reduced Instruction Set Computer

- Tradeoff:

- Execution time = *instructions per program* x *time per instruction*



RISC-V ISA

- RISC-V is modular, containing base and extensions
 - RV32I: base 32-bit instruction (arithmetic, load/store, control flow): 47 instructions
 - RV64I: additional instructions for 64-bit integer support: 14 instructions
 - M: integer multiplication and division: 8 instructions
 - A: atomic memory operations for synchronization: 11 instructions
 - F: single-precision floating-point operations: 25 instructions
 - D: double-precision floating-point operations: 25 instructions
 - Q: quad-precision floating-point operations: 27 instructions
 - C: compressed (16-bit) encodings for code density: 36 instructions
 - V: vector operations for parallel processing: 186 instructions
 - H: hypervisor support for virtualization: 2 instructions
 - Z: a group of "subextensions" and self-contained extensions (e.g. half-precision arithmetic, crypto, new vector operations, vector cryptography, cache prefetch hints, etc.)

Registers

- RISC-V (like MIPS) has 32 general purpose registers
 - In machine language: referenced as a 5-bit number
 - In assembly language: referenced as x0 to x31
 - Recall: MIPS assembly used \$0 to \$31
- Like MIPS, RISC-V registers have alternative names:
 - s0 to s11: (supposed to be) preserved across function calls
 - t0 to t6: not preserved across function calls
 - a0 to a7: function arguments and return values
 - ra, sp, gp, tp
 - zero

The Constant Zero

- RISC-V register 0 (zero) is the constant 0
 - Cannot be overwritten
- Useful for common operations
 - e.g., move between registers
add t2, s1, zero

Immediate Operands

- Constant data specified in an instruction

`addi s3, s3, 4`

- No subtract immediate instruction

- Use a negative constant

`addi s2, s1, -1`

Assembly Language Challenges

Having to load and store separately from compute and allocating registers

$f = (g + h) - (i + j);$

WRONG

```
add temp1, g, h
add temp2, i, j
sub f, temp1, temp2
```

CORRECT

```
lw s1,g
lw s2,h
lw s3,i
lw s4,j
add t0, s1, s2 # g+h
add t1, s3, s4 # i+j
sub s0, t0, t1
sw s0,f
```

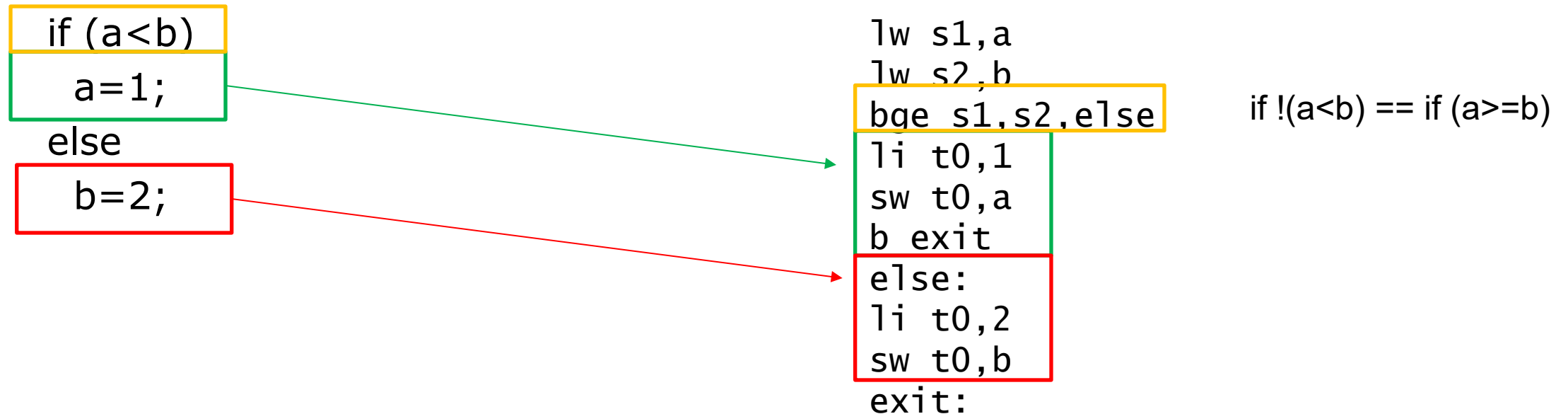
Assembly Programming Challenges

$a = (b + c) * d + b;$

```
lw    t0,b          # t0 = b
lw    t1,c          # t1 = c
add   t2,t0,t1       # t2 = (b + c)
lw    t3,d          # t3 = d
mul   t0,t2,t3       # t0 = (b + c) * d
add   t1,t0,t0       # t1 = t0 + b
sw    t1,a
```

Assembly Language Challenges

if-statement logic must be **complemented** when using conditional branches



Assembly Programming Challenges

No index for array access

```
int vals[100];  
for (int i = 0; i < 100; i++) sum += vals[i];
```

- There is no hardware support for indices in RISC-V

WRONG

```
li x1,0  
li x4,100  
loop: lw x3,vals[x1]  
      addi x1,x1,1  
      blt x1,x4,loop
```

CORRECT

```
li x1,0  
la x2,vals  
li x4,100  
loop: lw x3,0(x2)  
      addi x2,x2,4  
      addi x1,x1,1  
      blt x1,x4,loop
```

Assembly Programming Challenges

Keeping track of array addresses and loop counters separately

```
int *dst,*src;  
for (int i = 0; i < n; i++) dst[i] = src[i];
```

Assume array elements
are 4 bytes each

```
li      t0,0          # t0 = i  
la      t1,dst        # t1 = dst  
la      t2,src        # t2 = src  
lw      s0,n          # s0 = n  
loop:   bge   t0,s0,exit # if i >= n, then exit (pretest)  
slli    t3,t0,2       # t1 = i * 4  
add     t4,t1,t3      # compute &dst[i]  
add     t5,t2,t3      # compute &src[i]  
lw      t6,0(t5)      # t6 = src[i]  
sw      t6,0(t4)      # dst[i] = t6  
addi    t0,t0,1       # i = i + 1  
b       loop  
exit:
```

Assembly Programming Challenges

Can't compare to a constant using only one instruction

if (a < 0) return;

WRONG

```
blt x1,0,exit
```

CORRECT

```
li x2,0  
blt x1,x2,exit
```

RISC-V Assembly Instruction Types

1. arithmetic, logical, shift, comparison: 3-register

<mnemonic> <dest>,<src1>,<src2>

add x1,x2,x3

2. arithmetic, logical, shift, comparison: 2-register+immediate

<mnemonic> <dest>,<src1>,<imm>

addi x1,x2,10

3. branch

<mnemonic> <src1>,<src2>,<label>

blt x1,x2,loop

4. jump

<mnemonic> <label>

jal subr

5. load/store

<mnemonic> <reg>,label

<mnemonic> <reg>,<offset>(<base>)

lw x1,48(x2)

6. "coprocessor" instructions

<mnemonic> <reg1>,<imm>,<reg2>

csrrw rd,imm,rs1

Load/Store Instructions

- In assembly language:

```
var: .data
      .word      1, 2, 3
      .text
lw    x2, var      # loads value 1 (loads value 1)
la    x3, var
lw    x2, 0(x3)     # loads value 1 (same result as above)
lw    x2, 4(x3)     # loads value 2 (loads value 2)
addi  x3, x3, 8
lw    x2, 0(x3)     # loads value 3 (loads value 3)
addi  sp, sp, -4    # allocates 4 bytes on stack
sw    s0, 0(sp)     # pushes s0 to stack

lw    s0, 0(sp)     # pop from the stack
addi  sp, sp, 4
```

auipc x2, 0xnnnnnn # 20 bits
addi x2, x2, 0xnnn # 12 bits
lw x2, 0(x2)

Example Program 1

- C code:

```
int i,sum=0;
for (i=0;i<4;i++)
    sum+=vals[i];
```

```
                .data
vals:           .word  2,5,6,8

                .text
main:           li      t0,0           # sum
                li      t1,0           # i
                la      t2,vals
                li      t4,4
loop:           slli    t3,t1,2
                add     t3,t2,t3
                lw      t3,0(t3)
                add     t0,t0,t3       # sum+=vals[i]
                addi    t1,t1,1       # i++
                blt     t1,t4,loop
```

Example Program 2

- Accept console input instead of loading from memory

```
        .text
main:   li      t0,0          # sum
        li      t1,0          # i
        la      t2,vals
        li      t4,4
loop:   li      a7,5
        ecall
        add     t0,t0,a0      # sum+=vals[i]
        bne     a0,zero,loop
```

RISC-V Instructions

Basic arithmetic and bitwise logical:

<code>add rd, rs1, rs2</code>	<code># R[rd] = R[rs1] + R[rs2]</code>
<code>sub rd, rs1, rs2</code>	<code># R[rd] = R[rs1] - R[rs2]</code>
<code>and rd, rs1, rs2</code>	<code># R[rd] = R[rs1] & R[rs2]</code>
<code>or rd, rs1, rs2</code>	<code># R[rd] = R[rs1] R[rs2]</code>
<code>xor rd, rs1, rs2</code>	<code># R[rd] = R[rs1] ^ R[rs2]</code>

Shift instructions:

<code>sll rd, rs1, rs2</code>	<code># R[rd] = R[rs1] << R[rs2]</code>
<code>sra rd, rs1, rs2</code>	<code># R[rd] = (signed)R[rs1] >> R[rs2]</code>
<code>srl rd, rs1, rs2</code>	<code># R[rd] = (unsigned)R[rs1] >> R[rs2]</code>

Comparison instructions:

<code>slt rd, rs1, rs2</code>	<code># R[rd] = (signed)R[rs1] < (signed)R[rs2]</code>
<code>sltu rd, rs1, rs2</code>	<code># R[rd] = (unsigned)R[rs1] < (unsigned)R[rs2]</code>

RISC-V Instructions

Multiply instructions:

`mul rd, rs1, rs2`

`# R[rd] = R[rs1] * R[rs2]`

`mulh rd, rs1, rs2`

`# R[rd] = ((signed)R[rs1] * (signed)R[rs2]) >> 32`

`mulhu rd, rs1, rs2`

`# R[rd] = ((unsigned)R[rs1] * (unsigned)R[rs2] ) >> 32`

RISC-V Instructions

Basic arithmetic and bitwise logical:

```
addi rd, rs1, immediate  # R[rd] = R[rs1] + immediate
andi rd, rs1, immediate  # R[rd] = R[rs1] & immediate
ori  rd, rs1, immediate  # R[rd] = R[rs1] | immediate
xori rd, rs1, immediate  # R[rd] = R[rs1] ^ immediate
```

Shift instructions:

```
slli rd, rs1, immediate  # R[rd] = R[rs1] << immediate
srai rd, rs1, immediate  # R[rd] = (signed)R[rs1] >> immediate
srli rd, rs1, immediate  # R[rd] = (unsigned)R[rs1] >> immediate
```

RISC-V Instructions

Branch and Jump Instructions:

```
beq rs1, rs2, <label> # if R[rs1] == R[rs2] then branch to target
bne rs1, rs2, <label> # if R[rs1] != R[rs2] then branch to target
bge rs1, rs2, <label> # if (signed)R[rs1] > (signed)R[rs2] then branch to target
bgeu rs1, rs2, <label> # if (signed)R[rs1] >= (signed)R[rs2] then branch to target
blt rs1, rs2, <label> # if (signed)R[rs1] < (signed)R[rs2] then branch to target
bltu rs1, rs2, <label> # if (unsigned)R[rs1] < (unsigned)R[rs2] then branch to target

b <label>                # unconditionally branch to target (resolve to jal instruction)
jal rd, <label>          # R[rd] = return address (PC+4), jump to label
jalr rd, rs, imm         # R[rd] = return address (PC+4), PC = R[rs] + imm
```

RISC-V Instructions

Load and Store Instructions (not in course CPU):

```
lw rd, <label>          # R[rd] = Mem[label]
sw rs2, <label>          # Mem[label] = R[rs2]
```

Load Upper Immediate:

```
lui rd,imm (U-type)      # R[rd] = imm(20 bit) << 12
```

Fixed Point

- Fixed-point
 - Assume a decimal point at some location in a value:

1	0	1	1	0	1
---	---	---	---	---	---

As 6-bit integer: $2^5 + 2^3 + 2^2 + 2^0 = 32 + 8 + 4 + 1 = 45$

1	0	1.	1	0	1
---	---	----	---	---	---

As (6,3)-fixed format: $2^2 + 2^0 + 2^{-1} + 2^{-3} = 5 + .5 + .125 = 5.625$
 $= 5 + 5/8$

1	0.	1	1	0	1
---	----	---	---	---	---

As (6,4)-fixed format: $2 + 13/16 = 2.8125$

1	0.	1	1	0	1
---	----	---	---	---	---

45

2.8125

x

1	0.	1	1	0	1
---	----	---	---	---	---

45

2.8125

=

0	1	1	1.	1	1	1	0	1	0	0	1
---	---	---	----	---	---	---	---	---	---	---	---

2025

$7 + (128+64+32+8+1)/256$

$7 + 233/256 = 7.91015625$

Fixed Point Range

- Range:

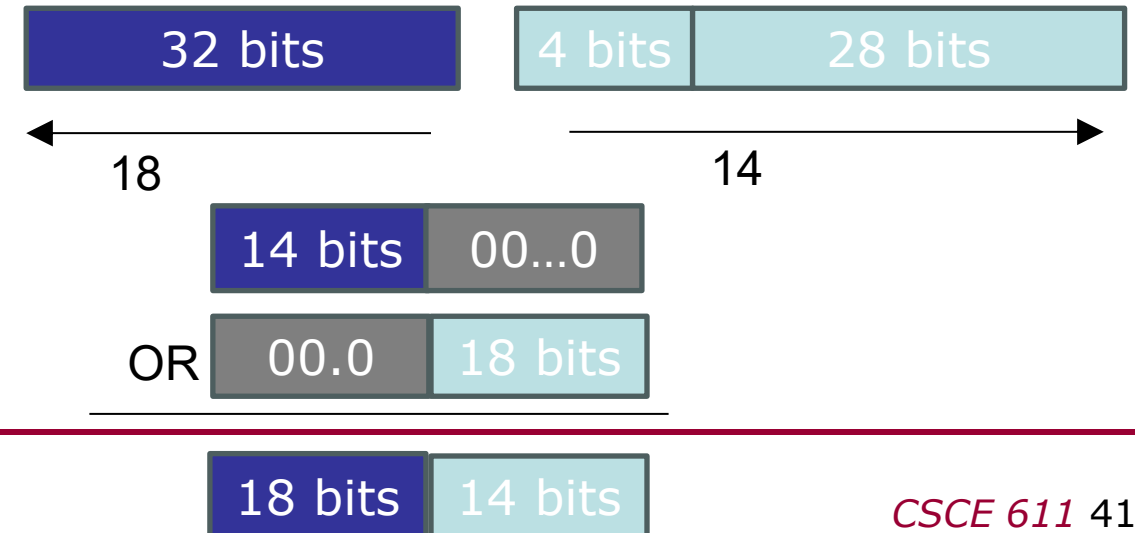
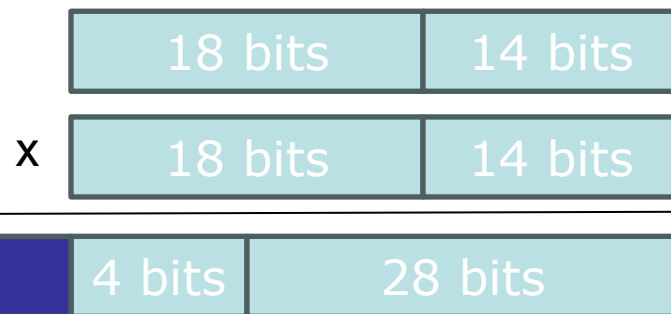
- Fixed point (signed): $[-2^{n-m-1}, 2^{n-m-1} - 1/2^m]$
 - Depends on # bits to the left of radix point
- Example: (32,23)
 - ULP (accuracy) = $2^{-23} \approx 10^{-7}$
 - Range = -2^8 to $(2^8 - 1/2^{23})$
 - Precision = 32 - # leading zeros
- Recall: floating point (approx.): $[-2^{maxexp}, 2^{maxexp}]$
 - Depends on exponent field size
 - Single precision:
 - ULP = $2^{exponent-23}$
 - Range = 2^{-127} to $2^{128} = 10^{-39}$ to 10^{38}
 - Precision = fixed at 24 bits

Fixed Point Operations

- Fixed point add/subtract
 - Before performing operation, ensure both operands have an equal number of fractional bits
 - If not, shift the value with the most fractional bits to the right by the difference in fractional bits
 - Example:
 - Assume value a is a (32,12) value and value b is a (32,10) value
 - $c = (a \gg 2) + b$
 - c is a (32,10) value

Fixed Point Multiply

- Multiplying values of types (n,f) and (n,f) will produce value of type $(2n,2f)$
- To convert back to original format, shift product to the right by f bits
- Example:
 - `uint32_t a,b,prod; // assume f fractional bits`
 - `prod = ((uint64_t)a * (uint64_t)b) >> f;`



Square Root

- Basic idea:
 - For input value **S**
 - Write a problem that solves equation $x^2 - S = 0$
 - Simplest solution: use binary search
 1. Choose initial guess for $X = 0.0$
 2. Choose initial **step**
 3. Repeat until $\text{step} == 0$:
 1. $\text{val} = x^2 - S$
 2. if $x^2 < S$ then $x = x + \text{step}$ else $x = x - \text{step}$
 3. $\text{step} = \text{step} / 2$

- Assume $S=2$

x	x ²	action	step
0	0	add	2
2	4	sub	1
1	1	add	.5
1.5	2.25	sub	.25
1.25	1.5625	add	.125
1.375	1.8906	add	.0625
1.4375	2.0664	sub	.03125
1.40625	1.9775	sub	0.01563
...			

Example: find sqrt(2)

- Use (32,14) format
- Initialize:
 - x_2 (guess) = 0 // set guess to 0, with 14 fractional bits
 - x_3 (step) = $256 \ll 14$ // set step to 256, with 14 fractional bits
- Loop:
 - $x_4 \leftarrow \text{lo}(x_2 * x_2)$ // square guess, get low bits (mul)
 - $x_5 \leftarrow \text{hi_u}(x_2 * x_2)$ // square guess, get high bits (mulhu)
 - $x_4 \leftarrow (x_4 \gg 14) \mid (x_5 \ll 18)$ // shift (hi,low) by 14 bits to right
 - if $x_4 == x_1$ then break // if exact, exit
 - if $x_4 < x_1$ then
 - $x_2 \leftarrow x_2 + x_3$ // otherwise, add step if too low
 - else
 - $x_2 \leftarrow x_2 - x_3$ // subtract step if too high
 - $x_3 \leftarrow x_3 \gg 1$ // half the step
 - if $x_3 == 0$ break // stop condition
 - goto loop